Convención de estilos y calidad

EcoHarmony Park | Ingeniería de Software | Grupo 04 Versión 4.0 | 21 de octubre de 2026

1. Propósito y aplicación

Esta convención establece las reglas de escritura, revisión y presentación del código del TP06. Se aplica al servicio Node.js, sus pruebas y el prototipo web en JavaScript. Las reglas locales prevalecen sobre las referencias generales. El equipo conserva una única fuente editable en este Markdown; el PDF es su representación para lectura y presentación.

Las reglas de negocio y los contratos de datos se describen en la especificación del caso de uso. La metodología y la trazabilidad de los ciclos de desarrollo se registran en la guía de trabajo y en Git. Estos documentos no se duplican dentro de la convención.

2. Herramientas y verificación

- JavaScript con módulos ES y Node.js 24. La versión se declara en engines y en .nvmrc.
- Las dependencias se resuelven mediante package-lock.json; la instalación reproducible usa npm ci.
- Prettier establece el formato y ESLint verifica las reglas estáticas mediante eslint.config.js.
- Vitest ejecuta las pruebas con describe, it, expect y beforeEach.
- Los comandos de control se ejecutan desde Implementacion:
  - npm run format: aplica el formato.
  - npm run format:check: comprueba el formato sin escribir.
  - npm run lint: comprueba las reglas estáticas.
  - npm test: ejecuta la suite completa.
  - npm run test:watch: ejecuta las pruebas en modo interactivo.
- Un cambio se considera verificable cuando formato, lint y pruebas terminan correctamente. Las modificaciones de presentación también requieren revisar el recorrido visual.

3. Escritura y nombres

- Archivos de texto en UTF-8, finales de línea LF y sangría de dos espacios, sin tabulaciones.
- Prettier usa 100 columnas como objetivo, comillas simples, punto y coma, paréntesis en funciones flecha y coma final en estructuras multilínea. Las URLs, expresiones regulares y cadenas indivisibles pueden superar ese ancho; la excepción no justifica comprimir una función.
- Clases en PascalCase; funciones, métodos, parámetros y variables en camelCase.
- Constantes de configuración y patrones regulares en SCREAMING_SNAKE_CASE.
- Los conceptos propios del dominio usan nombres en español, sin tildes en identificadores. Se conservan los nombres originales de APIs, eventos, bibliotecas y términos técnicos comunes.
- Las claves que forman contratos de datos, mensajes entre ventanas, atributos HTML y selectores CSS se conservan de manera consistente con sus consumidores. Un cambio de contrato se realiza de forma explícita y actualiza todos sus usos.
- Los módulos y utilidades usan archivos camelCase.js; los tests usan camelCase.test.js.
- Son nombres técnicos fijos package.json, package-lock.json, README.md, .gitignore, .nvmrc, eslint.config.js, .prettierrc.json, .prettierignore, index.html y styles.css.
- Una instrucción por línea y una declaración por línea, sin alineación mediante espacios.

4. Sintaxis, módulos y legibilidad

- Usar const por defecto y let cuando exista reasignación. No usar var.
- Usar comparaciones estrictas. Se admite valor == null únicamente para tratar null y undefined conjuntamente de forma intencional.
- Todas las estructuras de control llevan llaves, incluso cuando contienen una sola instrucción.
- Las funciones de módulo se declaran con nombre. Las funciones flecha se reservan para callbacks y funciones anidadas. Los métodos de objetos usan la sintaxis abreviada.
- Evitar ternarios anidados. Preferir retornos tempranos y condiciones que puedan leerse directamente.
- Usar literales de objetos y arreglos, y for...of para los recorridos imperativos.
- No usar eval, with, constructores Number/String/Boolean ni modificar prototipos nativos.
- Usar plantillas para interpolación y HTML multilínea. Las plantillas largas se dividen en fragmentos con nombres que expliquen la sección renderizada.
- Usar import/export con exportaciones nombradas. No mezclar require. export default se admite únicamente en configuraciones cuyo contrato técnico lo exige, como eslint.config.js. Los módulos de aplicación usan exportaciones nombradas. Las importaciones relativas incluyen .js; los módulos de Node.js usan el prefijo node:.
- Las dependencias entre módulos son acíclicas. Los valores exportados no se alteran desde sus consumidores. Los objetos de configuración compartida se congelan.
- No crear capas, clases ni funciones auxiliares que solo oculten una llamada trivial. Un refactor mejora la lectura sin introducir reglas nuevas.

5. Contratos, validación y efectos

- Toda entrada externa se trata como desconocida. Validar presencia, estructura y tipo antes de usarla; documentar las conversiones explícitas que admita su contrato.
- Distinguir campo omitido, null, cadena vacía, espacios, cero y false. No resolverlos mediante un único control de valor verdadero/falso.
- Usar ?? para valores predeterminados por ausencia. Los reemplazos de textos vacíos para presentación se expresan mediante una comprobación explícita.
- Validar números enteros con Number.isInteger; no convertir implícitamente texto a número. Los documentos personales se conservan como texto.
- No mutar la solicitud recibida ni exponer referencias modificables al almacenamiento.
- Separar comprobaciones sin efectos de registro, consumo de cupos y generación de confirmación.
- El servicio usa errores de tipo Error con categoría identificable y campo afectado; cuando corresponda, incluye el número de participante. No lanzar textos ni objetos sueltos.
- No transformar un fallo en un éxito ficticio ni mostrar stack traces al visitante. Todo catch que descarte un error debe explicar el motivo y limitarse al caso esperado.
- Usar async/await. Las promesas se esperan, se retornan o se atienden explícitamente en el límite de un evento; no dejar rechazos sin manejar.
- En el almacenamiento en memoria, validar cupos/conflictos y registrar sin intercalar await. Esta garantía corresponde al proceso y adaptador utilizados; no acredita una transacción de BD.
- El instante actual proviene de un reloj inyectable. El reloj del sistema es un adaptador separado. Construir una fecha a partir de un dato explícito no equivale a consultar la hora actual.
- Usar una zona horaria explícita en el calendario y la presentación, independiente del dispositivo.

- Las edades, capacidades, duración y ventana temporal se obtienen de configuración compartida. Los fixtures de tests declaran sus valores esperados para no reproducir la implementación.

6. Documentación del código

- Las clases, funciones, constantes de configuración y métodos públicos tienen JSDoc útil.
- Describir propósito, parámetros, resultado y errores de dominio mediante @param, @returns y @throws cuando correspondan. Definir estructuras de datos con @typedef cuando se reutilicen.
- Los comentarios internos usan // o /* */ y explican decisiones o restricciones. No repetir mecánicamente el nombre de la instrucción ni exigir documentación de helpers triviales.
- Los datos externos se validan en ejecución; JSDoc y la información del editor no sustituyen esas comprobaciones.

7. Convención de pruebas

- Nombres en español con el patrón debe [resultado] cuando [condición].
- Estructura AAA: preparar, ejecutar y comprobar. Los comentarios de las tres fases se usan cuando ayudan a leer el escenario; no son obligatorios en una validación breve.
- Probar interfaces públicas y efectos observables. No probar métodos privados, getters triviales ni detalles internos sin comportamiento propio.
- En validaciones simples, agrupar alternativas representativas mediante expect explícitos. No parametrizar ni multiplicar variantes equivalentes. Los límites y causas diferentes sí se cubren.
- Cada test representa una regla. Los fixtures pequeños pueden construir una solicitud válida, pero no deben esconder el dato relevante ni la acción que se quiere comprobar.
- Repositorio, reloj y bandeja de correo aislados por test; datos ficticios y tiempos controlados.
- En rechazos, comprobar el error y la ausencia de efectos indebidos sobre registro, cupos y correo. En éxitos, comprobar los datos guardados y la confirmación correspondiente.
- Verificar un QR con un decodificador independiente, sin comparar bytes PNG exactos.
- Las pruebas concurrentes inician solicitudes antes de esperar su resultado; no usan sleeps.
- Un resultado verde demuestra los escenarios ejecutados; no acredita por sí solo metodología TDD ni sustituye las reglas del dominio.

8. Presentación y accesibilidad

- HTML, CSS y JavaScript con diseño responsive y lectura prioritaria en celular.
- Fuente Lato, servida localmente, con alternativas del sistema.
- Tokens de interfaz: verde oscuro #386641, verde medio #6a994e, verde claro #a7c957, crema #f2e8cf y rojo #bc4749. Los controles, textos y superficies utilizan esta paleta.
- Las transparencias de estos colores se reservan para bordes, sombras, fondos decorativos y superposiciones. El texto de lectura usa verde oscuro sólido sobre crema o crema sobre verde oscuro.
- Fotografías, videos y gráficos técnicos de QR conservan sus colores propios.
- Contraste mínimo de 4,5:1 en texto normal y 3:1 en texto grande. Verificar el contraste real sobre imágenes o transparencias. Verde medio, verde claro y rojo no se usan como texto pequeño sobre crema. Un estado nunca se comunica únicamente por el color.
- Controles etiquetados, foco visible y navegación directa por teclado. Las acciones principales tienen al menos 44 por 44 píxeles CSS y no dependen de hover.

- La presentación de datos y errores respeta el contrato del componente que los produce. El código de presentación no duplica ni reemplaza las reglas del servicio.
- Los modales tienen nombre accesible, cierre por teclado y restauración del foco.
- Las transiciones de controles son breves. Se admiten videos y movimiento ambiental decorativo que no oculten controles, retrasen la navegación ni desplacen campos.
- Respetar prefers-reduced-motion; pausar medios y ambiente al ocultar la pestaña, abrir un modal o escribir. Con movimiento reducido, conservar el contenido y el recorrido mediante imágenes.
- El prototipo visual conserva un estado propio e independiente del servicio. Sus etiquetas, atributos y mensajes de presentación no alteran los contratos de inscripción.

9. Mantenimiento y referencias

El Markdown es la fuente de esta convención. Toda actualización identifica su versión y fecha, y regenera el PDF para conservar la correspondencia entre ambos formatos. Las excepciones se justifican de forma específica y no anulan el resto de las reglas. Los cambios de formato y refactor se distinguen de los cambios funcionales y se verifican con las mismas herramientas.

Referencias técnicas:

- Google JavaScript Style Guide: https://google.github.io/styleguide/jsguide.html
- Node.js, ECMAScript modules: https://nodejs.org/api/esm.html
- ESLint, configuración: https://eslint.org/docs/latest/use/configure/
- Prettier, opciones: https://prettier.io/docs/options
- W3C, contraste: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html